

Dynamic Games and Backward Induction

2026-03-11

From Simultaneous to Sequential

We studied **static games**: both players choose at the same time (or at least without knowing what the other chose).

But many real interactions have a **sequence**: one person moves, then the other responds.

- Negotiations: one side makes an offer, the other accepts or rejects
- Chess, checkers, tic-tac-toe: players alternate moves
- Legislation: a bill is proposed, then amended, then voted on

What Changes?

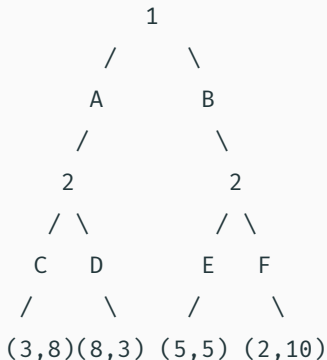
In a simultaneous game, you have to anticipate what the other player will do.

In a sequential game, the later mover can *see* what happened before deciding.

This might seem like it always helps the later mover (more information), but it turns out the **first mover** often has the advantage—because they can shape the situation the later mover faces.

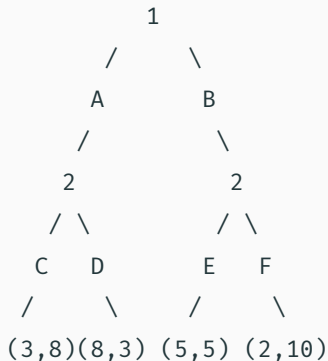
Game Trees

We represent sequential games with **game trees** (also called the “extensive form”).



- Nodes = decision points (labeled with who decides)
- Branches = available actions
- Leaves = outcomes with payoffs (Player 1, Player 2)

Reading the Tree



1. Player 1 chooses A or B
2. Player 2 *sees* what Player 1 chose
3. If Player 1 chose A: Player 2 picks C or D
4. If Player 1 chose B: Player 2 picks E or F

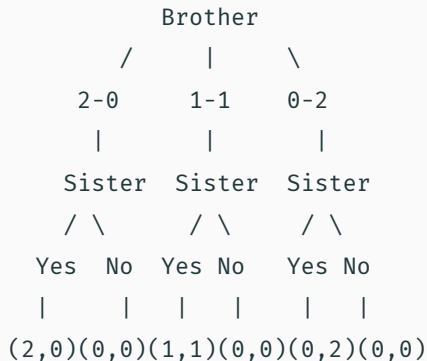
Player 2 can have a different plan for each scenario.

The Sharing Game

A brother and sister are dividing two identical presents.

1. The brother proposes a split: he keeps both (2-0), they each keep one (1-1), or she keeps both (0-2).
2. The sister can **accept** or **reject** the proposal.
3. If she accepts, they get the proposed split. If she rejects, *nobody gets anything*.

The Game Tree



What will the sister do at each decision point?

What should the brother propose, knowing what the sister will do?

Backward Induction

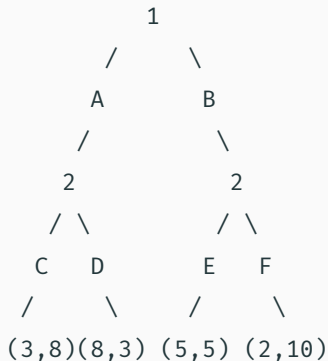
How do you solve a sequential game? **Start at the end and work backwards.**

This is called **backward induction**, and it's one of the most important ideas in game theory.

The logic:

1. Look at the *last* decision point. What will that player do? (Easy: pick whatever gives them the highest payoff.)
2. Now go back one step. The earlier player *knows* what the later player will do. So they can predict the outcome of each choice.
3. Keep going until you reach the first move.

Example

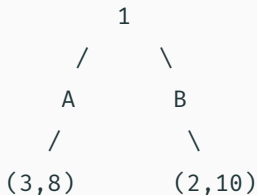


Step 1: At Player 2's decision points:

- After A: Player 2 prefers C (gets 8) over D (gets 3). So Player 2 will choose C.
- After B: Player 2 prefers F (gets 10) over E (gets 5). So Player 2 will choose F.

Example (cont.)

Knowing Player 2's choices, the tree simplifies to:



Step 2: Player 1 prefers A (gets 3) over B (gets 2). So Player 1 chooses A.

Result: Player 1 plays A, Player 2 plays C, outcome is **(3, 8)**.

Notice: Player 1 gets only 3 even though outcomes giving Player 1 as much as 8 exist in the tree. But Player 1 can't force those outcomes—Player 2 gets to respond.

Back to the Sharing Game

Brother					
/				\	
2-0		1-1		0-2	
Sister		Sister		Sister	
/ \		/ \		/ \	
Yes	No	Yes	No	Yes	No
(2,0)		(0,0)		(1,1)	
(0,0)		(0,2)		(0,0)	

Backward induction: Sister accepts 2-0 (2-0: $0 > 0$, so she's actually *indifferent*... but she accepts 1-1 and 0-2 for sure).

So the brother should propose... **2-0**?

Does that feel right?

This is essentially the **ultimatum game**, one of the most-studied games in experimental economics.

What actually happens when people play:

- Proposers typically offer **40-50%** of the total (not the minimum)
- Responders frequently **reject** offers below about 20-30%
- This pattern holds across cultures (with some variation)

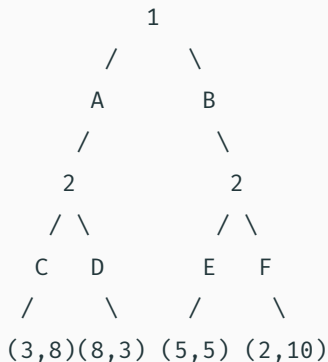
People seem to care about **fairness**, not just the payoff numbers. This is a real challenge for simple game theory.

Subgame-Perfect Equilibrium

We can still find Nash equilibria in sequential games—just convert the tree to a payoff matrix.

But there's a problem. Some Nash equilibria involve **threats that aren't credible**.

An Example of an Empty Threat



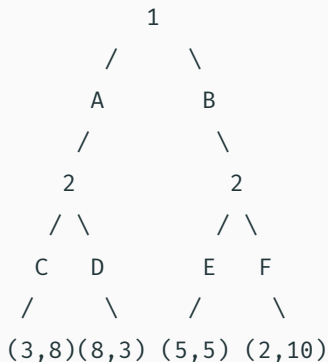
Here's a Nash equilibrium

- Player 1 plays A, Player 2 plays “C if A, **E if B.**”

Player 2 is essentially saying: “Don’t play B, or I’ll play E instead of F.”

But if Player 1 *did* play B, would Player 2 really choose E (payoff 5) over F

Why This Matters



Here's another Nash equilibrium:

- Player 1 plays B, Player 2 plays “D if A, F if B.”

Player 1 gets 2, Player 2 gets 10.

Player 2 is threatening: “If you play A, I’ll play D (bad for you).”

To rule out empty threats, we need a stronger concept: **subgame-perfect equilibrium** (SPE).

The idea: a strategy profile is subgame-perfect if it's a Nash equilibrium in *every* subgame, including subgames that wouldn't actually be reached.

A **subgame** is any part of the game tree that could be a game on its own (starting from any decision node and including everything below it).

Here's the key insight:

*In games where players take turns and can see everything that's happened, **backward induction always finds a subgame-perfect equilibrium.***

In our example, the only SPE is:

- Player 1 plays A, Player 2 plays “C if A, F if B.”

Player 2 plays the best response at every point in the tree, even at points that won't be reached. That's what makes the threat credible.

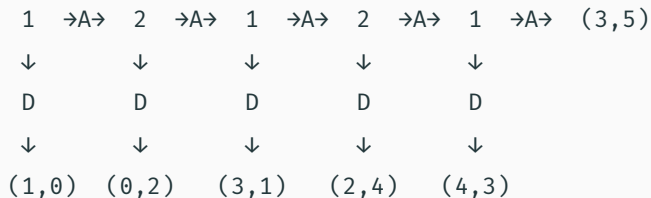
This connects to a lot of real-world reasoning:

- **Deterrence:** A country threatens nuclear retaliation. But would they actually follow through? (Compare: Schelling on limited warfare.)
- **Parenting:** “If you do that one more time, we’re going home.” Will you actually leave?
- **Business:** “If you enter our market, we’ll slash prices.” Would a company really hurt its own profits to drive out a competitor?

Subgame perfection says: only threats you’d actually carry out should be taken seriously.

The Centipede Game

A Puzzle for Backward Induction



Players alternate choosing: go **across** (continue) or go **down** (end the game). The payoffs generally grow as you go across.

What Does Backward Induction Say?

Start from the end:

- **Last move** (Player 1): Down gives $(4,3)$, Across gives $(3,5)$. Player 1 goes down.
- **Second to last** (Player 2): Knowing Player 1 will go down next, Player 2 compares: Down gives $(2,4)$, continuing leads to $(4,3)$. Player 2 goes down.
- **Working backwards**: The same logic applies at every node.

Backward induction says Player 1 should go down immediately, getting $(1,0)$.

But the “cooperative” outcome $(3,5)$ is much better for both players!

What Actually Happens

In experiments, people almost never go down at the first node. They typically continue for several rounds.

This creates a puzzle:

- Are the experimental subjects irrational?
- Or is backward induction too demanding here?
- What should Player 2 think if Player 1 *doesn't* go down at the start?

The theory said they would, so if they didn't, maybe the theory's assumptions don't apply...

This is one of the deepest philosophical puzzles in game theory, and connects to questions about **common knowledge of rationality** and what to believe when someone does something your theory says they wouldn't.

Imperfect Information

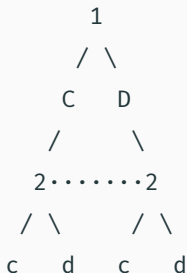
So far, the later player has always been able to see what the earlier player did. That's **perfect information**.

But many real games involve **imperfect information**: you know it's your turn to move, but you don't know exactly what happened before.

Examples:

- Poker: you don't see the other player's cards
- Sealed-bid auctions: you don't see other bids
- The Prisoner's Dilemma: you choose "simultaneously" (neither sees the other's choice)

We represent uncertainty by grouping together decision nodes that a player **can't tell apart**.



The dotted line means: Player 2 can't tell which node they're at. They know it's their turn, but not whether Player 1 chose C or D.

This is exactly the **Prisoner's Dilemma** in tree form! Simultaneous games are just sequential games where the second player has no information about the first player's move.

With perfect information, backward induction works beautifully. Start at the leaves, work up.

With imperfect information, there's a problem: the player at an information set has to choose **one action for the whole set**. They can't condition their choice on which node they're actually at.

This means subgame perfection doesn't directly apply in the same clean way. Game theorists have developed more sophisticated concepts (like **sequential equilibrium**) for these cases, but they get complicated quickly.

For this class, the key takeaway is:

1. When players move in sequence and can see everything: use **backward induction** to find the subgame-perfect equilibrium
2. This rules out **non-credible threats**
3. When there's **uncertainty** about what happened earlier, the analysis gets much more complex
4. Many real-world strategic situations involve *some* sequential moves and *some* hidden information

Connecting the Threads

Static vs. Dynamic Games: Summary

	Static (Normal Form)	Dynamic (Extensive Form)
Representation	Payoff matrix	Game tree
Timing	Simultaneous	Sequential
Key solution	Nash equilibrium	Subgame-perfect eq.
Main idea	Best response	Backward induction
Weakness	Multiple equilibria	Extreme rationality assumptions

Game theory gives us these clean, precise predictions. But real humans:

- Care about fairness (ultimatum game rejections)
- Cooperate even when theory says defect (centipede game)
- Are willing to trust and reciprocate (trust game)
- Use backward induction *sometimes*, but not when it leads to absurd conclusions

This creates some challenges for what to do about

Next time: **Axelrod's Iterated Prisoner's Dilemma Tournaments**

The question: if you play the Prisoner's Dilemma *many* times against the same opponent, what strategy works best?

- Think about what strategy *you* would submit to a tournament.
- Should you ever cooperate? When?
- Does it matter whether the game has a known endpoint?

Read about Axelrod's tournaments.

And here's a question to think about:

Backward induction says you should defect in the last round of a repeated Prisoner's Dilemma (since there's no future to incentivize cooperation). But if you'll defect in the last round, your opponent knows that, so they'll defect in the second-to-last round... and so on. Does this argument really prove you should always defect?